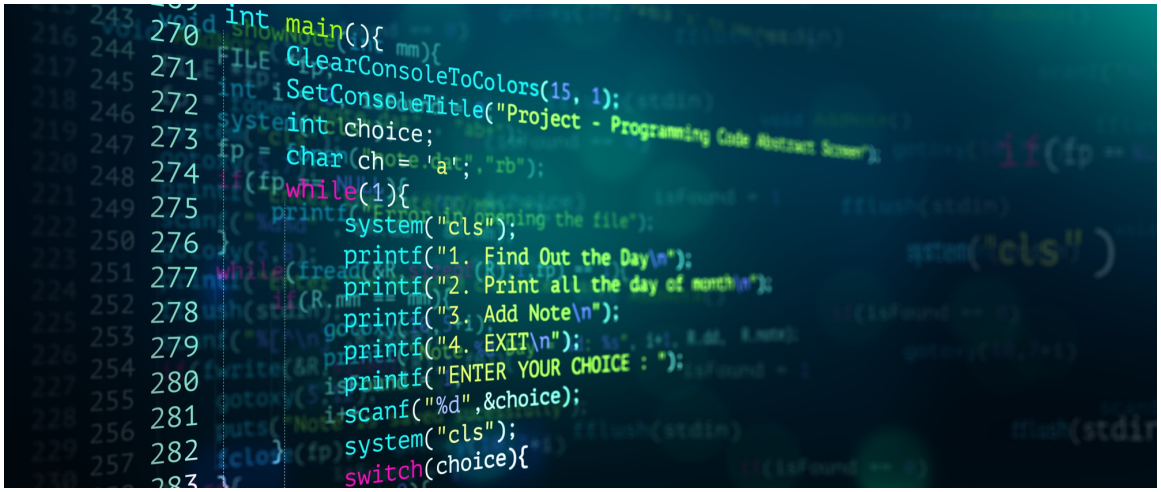

C - exercise book

Piotr Jastrzębski

2026-03-21

Table of contents

C - exercise book 3

I Fundamentals of the C Language 4

1 Input and Output Operations 5

2 Arithmetic Operators 6

3 Conditional statements, conditional operator 7

4 Bitwise operators 9

5 Debugging - basics 10

6 Loops 14

7 Functions 16

Bibliography and Other Problem Sets 18

C - exercise book

Here you will find a collection of programming exercises in the C language. The inspiration was to compile tasks created during teaching classes carried out at the Faculty of Mathematics and Computer Science of the University of Warmia and Mazury in Olsztyn.

Some of the exercises have been machine-translated into English. In case of any ambiguities, please get in touch via email.

Part I

Fundamentals of the C Language

1 Input and Output Operations

1. Write a programme that asks the user to enter a single integer, then displays it on the screen.
2. Create a programme that reads two floating-point numbers from the user and prints their difference.
3. Design an application that asks the user for their year of birth, then prints the previous year (one less).
4. Write a programme that reads three integers from the user and prints their average as a floating-point value.
5. Create a programme that asks the user to enter two letters (characters), then prints them in reverse order.
6. Write a programme that asks the user to enter a single floating-point number, then doubles its value and prints the result.
7. Create a programme that reads a floating-point number from the user representing an amount in dollars, then prints how much that is in euros, using a fixed exchange rate (e.g. 1 dollar = 0.85 euros).
8. Write a programme that displays the following text on the screen: `This is a quote: "I often use the C language."`. Ensure that the quotation marks are correctly displayed as part of the string.
9. Create a programme that prints a Windows folder path to standard output, e.g. `C:\Program Files\MyApplication\` (hardcoded, without retrieving anything from the system) or `C:\\Program Files\\MyApplication\\`.
10. Design a programme that demonstrates how to print the following text on the screen: `Special characters: \t (tab), \n (newline), % (percent), \\ (backslash).`
11. Write a programme that reads two rational numbers from standard input representing the lengths of the legs of a right-angled triangle. Then calculate and display the length of the hypotenuse.
12. Write a programme that reads an integer from standard input and prints its absolute value to standard output.
13. Write a programme that reads a floating-point number from standard input and prints its absolute value to standard output.
14. Find an example and display on standard output a case where the addition of floating-point numbers is not associative.

2 Arithmetic Operators

1. Write a programme that calculates the remainder of dividing the sum of two integers by a third integer.
2. Create a programme that calculates the difference of squares of two given integers.
3. Write a programme that calculates the product of the difference of two integers and a third integer.
4. Design a programme that calculates the geometric mean of the absolute values of three given integers.
5. Write a programme that calculates the square of the sum of two given integers.
6. Create a programme that calculates the sum of squares of three given integers.
7. Write a programme that calculates how many times one given integer fits into another given integer.
8. Design a programme that calculates the square of the difference of two given integers.
9. Write a programme that calculates the quotient of the sum of two integers divided by their difference.
10. Create a programme that calculates the sum of three consecutive integers, starting from a given integer.
11. Find the arithmetic mean of three floating-point numbers.
12. Calculate the value of the expression $\frac{1}{\frac{1}{a}+\frac{1}{b}+\frac{1}{c}}$, where a, b, c are non-zero.
13. Calculate the value of e^x for small values of x, e.g. $|x| < 0.01$, approximating e^x as $1 + x$, without using exponential functions.
14. Calculate the area of a triangle with sides a, b and the angle C between them in degrees, using the formula $0.5 \cdot a \cdot b \cdot \sin C$, assuming $\sin C \approx C$ for small angles in radians.
15. Calculate $\sqrt[3]{x}$ for small values of x, e.g. $|x| < 0.01$, approximating $\sqrt[3]{x} \approx 1 + \frac{1}{3}x$, without using any root functions.

3 Conditional statements, conditional operator

In the following exercises, use `if`, `if else`, or `switch case` as the conditional statement. The conditional operator refers to `?`.

1. Write a programme that asks the user to enter an integer. The programme should display whether the entered number is positive, negative, or equal to zero.
2. Write a programme that takes two integers from the user and displays the larger of the two.
3. Write a programme that asks the user to enter a grade on a scale from 1 to 5. The programme should display a description of the grade: fail (1), poor (2), satisfactory (3), good (4), very good (5). For a number outside the range, the programme should display an error message.
4. Write a programme that asks the user to enter three distinct integers and displays the smallest of them.
5. Write a programme that asks the user for a year and checks whether the given year is a leap year. A leap year is one that is divisible by 4 but not divisible by 100, unless it is also divisible by 400.
6. Write a programme that takes two integers from the user and displays whether the sum of both numbers is even or odd.
7. Write a programme that takes three floating-point numbers `a, b, c` from the user. Treat them as the coefficients of a quadratic equation $ax^2 + bx + c = 0$. Print all solution variants of this equation to standard output.
8. Write a programme that takes two integers as input from the user and uses the conditional operator to find and display the larger of the two.
9. Write a programme that takes three integers as input from the user and uses the conditional operator to find and display the smallest of them.
10. Using the conditional operator `?`, write a programme that takes a single integer from the user and displays "even" or "odd" depending on the value of the number.
11. Create a programme that asks the user to enter two floating-point numbers and uses the conditional operator to display which of them is larger, or whether they are equal to two decimal places.

12. Using the conditional operator, write a programme that asks the user to enter a grade on a scale from 0 to 100 and displays “Pass” if the grade is greater than or equal to 51, or “Fail” otherwise.

13. Write a programme that takes a year as input from the user and uses the conditional operator to check whether it is a leap year. The programme should display “Leap year” or “Not a leap year” depending on the result.

4 Bitwise operators

1. Swap the values of two integer variables without using an additional variable.
2. Check whether an integer is even or odd using bitwise operators.
3. Determine the value of the bit at a specified position in an integer.
4. Clear the value of the bit at a specified position in an integer.
5. Invert the values of all bits in an integer.
6. Shift the bits of an integer a specified number of positions to the left.
7. Shift the bits of an integer a specified number of positions to the right.
8. Determine the XOR of two integers.

5 Debugging - basics

1. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>

int main() {
    int a = 10; // a = , b =
    int b = 5;  // a = , b =
    a = a + b;  // a = , b =
    b = a - b;  // a = , b =
    a = a - b;  // a = , b =
    b = a * b;  // a = , b =
    a = b / a;  // a = , b =
    b = a << 2; // a = , b =
    a = b >> 1; // a = , b =
    b = a & b;  // a = , b =
    a = a ^ b;  // a = , b =
    b = ~a;     // a = , b =
    return 0;
}
```

2. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>

int main() {
    int a = 10; // a = , b = , c =
    int b = 5;  // a = , b = , c =
    int c = 0;  // a = , b = , c =
    a = a + b;  // a = , b = , c =
    b = a - b;  // a = , b = , c =
    a = a - b;  // a = , b = , c =
```

```
    c = a;       // a = , b = , c =
    a = b * c;   // a = , b = , c =
    b = a / c;   // a = , b = , c =
    c = b << 2;  // a = , b = , c =
    b = c >> 1;  // a = , b = , c =
    a = b & c;   // a = , b = , c =
    c = a ^ b;   // a = , b = , c =
    b = ~c;      // a = , b = , c =
    return 0;
}
```

3. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>
int main() {
    int x = 7;     // x = , y = , z =
    int y = 3;     // x = , y = , z =
    int z = 12;    // x = , y = , z =
    x = x * 2;     // x = , y = , z =
    y = y + x;     // x = , y = , z =
    z = z - y;     // x = , y = , z =
    x = z / 2;     // x = , y = , z =
    y = x % 3;     // x = , y = , z =
    z = x + y;     // x = , y = , z =
    x = z++;       // x = , y = , z =
    y = ++x;       // x = , y = , z =
    z = y--;       // x = , y = , z =
    return 0;
}
```

4. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>
int main() {
    int p = 15;    // p = , q = , r =
    int q = 6;     // p = , q = , r =
    int r = 1;     // p = , q = , r =
    p = p | q;     // p = , q = , r =
```

```
    q = p & 10;    // p = , q = , r =
    r = r << 3;    // p = , q = , r =
    p = p ^ r;     // p = , q = , r =
    q = q >> 1;    // p = , q = , r =
    r = ~p;        // p = , q = , r =
    p = q | r;     // p = , q = , r =
    q = p & r;     // p = , q = , r =
    r = r ^ q;     // p = , q = , r =
    return 0;
}
```

5. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>
int main() {
    float a = 10.5;  // a = , b = , c =
    float b = 2.0;   // a = , b = , c =
    float c = 0.5;   // a = , b = , c =
    a = a / b;       // a = , b = , c =
    b = b * 3;       // a = , b = , c =
    c = a + b;       // a = , b = , c =
    a = c - a;       // a = , b = , c =
    b = b / 2;       // a = , b = , c =
    c = a * b;       // a = , b = , c =
    a = c + 0.5;     // a = , b = , c =
    b = a - c;       // a = , b = , c =
    c = a / b;       // a = , b = , c =
    return 0;
}
```

6. Below is code written in C. Some lines contain comments. Your task is to fill in the values of the corresponding variables after each line of code is executed. Adding new lines or disrupting the code structure means the task was done incorrectly.

```
#include <stdio.h>
int main() {
    int m = 20;    // m = , n = , k =
    int n = 4;     // m = , n = , k =
    int k = 2;     // m = , n = , k =
    m += n;        // m = , n = , k =
```

```
    n -= k;        // m = , n = , k =
    k *= 3;        // m = , n = , k =
    m /= 2;        // m = , n = , k =
    n += m;        // m = , n = , k =
    k %= n;        // m = , n = , k =
    m <<= 1;       // m = , n = , k =
    n >>= 2;       // m = , n = , k =
    k ^= m;        // m = , n = , k =
    return 0;
}
```

6 Loops

The tasks should be solved without arrays, strings, pointers, or built-in math functions. Do not create your own functions either.

1. Write a program that displays all integers from 1 to 100 using a `for` loop.
2. Create a program that asks the user to enter an integer `n`, and then displays the sum of all integers from 1 to `n` using a `while` loop.
3. Write a program that reads integers from the user until zero is entered, and then displays the sum of all positive numbers and the sum of all negative numbers, using a `do-while` loop.
4. Create a program that calculates and displays the factorial of a non-negative integer entered by the user, using a `for` loop.
5. Write a program that displays the first 10 numbers of the Fibonacci sequence using a `while` loop.
6. Write a program that asks the user to enter a positive integer `n`, and then calculates and displays $\lfloor \sqrt{n} \rfloor$ (the integer part/floor of the square root).
7. Write a program that asks the user to enter a positive integer `n`, and then calculates and displays $\lceil \sqrt{n} \rceil$ (the ceiling of the square root).
8. Write a program that asks the user to enter 10 positive integers and arranges them into a sequence $a_1, \dots, a_{10}$. Calculate and display how many elements of the sequence satisfy the inequality $a_k < \frac{a_{k-1}+a_{k+1}}{2}$ for $1 < k < 10$.
9. Write a program that asks the user to enter a positive integer n, and then n positive integers and arranges them into a sequence $a_1, \dots, a_n$. Calculate and display how many elements of the sequence satisfy the inequality $a_k < \frac{a_{k-1}+a_{k+1}}{2}$ for $1 < k < n$.
10. Write a program that checks the divisibility of a number n by all numbers from 2 to n/2. The program should print all divisors of that number. Read the value of n from the user.
11. Write a program that reads an integer n and prints all its digits from the end (starting from the units digit) and calculates their sum.
12. Write a program implementing the Euclidean algorithm in an iterative version using subtraction. The program should read two natural numbers a and b, and then print their greatest common divisor.

13. Write a program implementing the Euclidean algorithm in an iterative version using division. The program should read two natural numbers a and b, and then print their greatest common divisor.

14. Write a program that checks whether a given number n is a prime number. The program should check all potential divisors from 2 to the square root of n (use a loop to calculate the square root). Print an appropriate message.

15. Write a program simulating change-making. The program should read the amount to be given as change and print how many coins of denominations 5, 2, and 1 should be used to minimize the total number of coins. Print the number of coins of each type.

16. Write a program that determines the approximate value of the square root of a number a using Newton's method (iterative). The initial approximation is `x = a/2`, and subsequent values are calculated using the formula `x_{i+1} = (x_i + a/x_i)/2`. Perform 10 iterations of the algorithm and print the final result.

17. Write a program that reads from the user the degree of a polynomial, the coefficients (from the highest power to the constant term), and the value of x. Calculate the value of the polynomial using Horner's method, utilizing a loop. Display the result of the calculations.

7 Functions

Solve the exercises without using arrays, strings, pointers, or built-in math functions. You may create additional helper functions for each exercise. For each function described in the exercise, provide a test case (call it at least once with a valid argument). The exercises use the `camelCase` naming convention. Pay attention to whether the function should return a value or print it.

1. Write a function `sumTwoNumbers` that takes two integers as arguments. The function should print the sum of the numbers passed as arguments. Create a test case.
2. Write a function `calculateAbsoluteValue` that takes a floating-point number as an argument. The function should return the absolute value of the number passed as an argument. Create a test case.
3. Write a function `calculateFactorial` that takes a non-negative integer as an argument. The function should return the factorial of the number passed as an argument, computed using a non-recursive method. Create a test case.
4. Write a function `sumNumbers` that takes a positive integer n as an argument. The function should return the sum of numbers from 1 to n inclusive. Create a test case.
5. Write a function `sumSquares` that takes a positive integer n as an argument. The function should return the sum of squares of numbers from 1 to n inclusive. Create a test case.
6. Write a function `calculatePowerOfTwo` that takes an integer n as an argument. The function should return the value of 2^n. Create a test case.
7. Write a function `calculateSquareRootFloor` that takes a non-negative integer n as an argument. The function should return the integer part of the square root of n. Create a test case.
8. Write a function `countFunctionCalls` that takes no arguments. The function should count and print to standard output the number of times it has been called since the program started. Create a test case. Use static variables.
9. Write a function `calculateFactorialRecursively` that takes a non-negative integer n as an argument. The function should return the factorial of n, computed using a recursive method. Create a test case.

10. Write a function `calculateFibonacciRecursively` that takes a non-negative integer n as an argument. The function should return the n-th term of the Fibonacci sequence, computed using a recursive method. Create a test case.

11. Write a function `calculateArithmeticSequenceRecursively` that takes two positive integers as arguments: n (the index of the sequence term to compute) and d (the common difference of the arithmetic sequence), assuming the initial term a_1 equals 1. The function should return the n-th term of the arithmetic sequence, computed using a recursive method. Create a test case.

12. Write a function `calculateGeometricSequenceRecursively` that takes two positive integers as arguments: n (the index of the sequence term to compute) and d (the common ratio of the geometric sequence), assuming the initial term a_1 equals 1. The function should return the n-th term of the geometric sequence, computed using a recursive method. Create a test case.

13. Write a function `calculate13` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = 2f(n-1)+3$, where $f(1) = 1$. Create a test case.

14. Write a function `calculate14` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = 3f(n-1)-1$, where $f(1) = 2$. Create a test case.

15. Write a function `calculate15` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = f(n-1)+2f(n-2)$, where $f(1) = 1$ and $f(2) = 2$. Create a test case.

16. Write a function `calculate16` that takes a positive integer n as an argument. The function should return the value defined by $f(n) = 2f(n-1)+3f(n-2)$, where $f(1) = 2$ and $f(2) = 3$. Create a test case.

17. Write a recursive function `calculate17` that takes a non-negative integer n as an argument. The function should return the value defined by $f(n) = f(0)+f(1)+...+f(n-1)$, where $f(0) = f(1) = 1$. Create a test case.

18. Write a recursive function `calculateGCD` that takes two positive integers n and m as arguments. The function should return the greatest common divisor (GCD) of these numbers using the Euclidean algorithm. Create a test case.

Bibliography and Other Problem Sets

Krzaczkowski, Jacek. 2011. *Zadania z Programowania w Języku c/c++, Cz. I.* Instytut Informatyki UMCS Lublin.